Experiment 6: Create Dynamic Dashboard Using Chart.js

Aim

To build an interactive dashboard that visualizes ML model predictions and performance metrics using Chart.js library.

Software Requirements

- Python 3.10+
- Flask
- pymongo
- MongoDB Atlas
- Chart.js (CDN)

Theory

Data visualization is crucial for understanding ML model behavior and prediction patterns. Chart.js is a popular JavaScript library for creating responsive, animated charts. In this experiment, we'll store prediction history in MongoDB and visualize it using various chart types (bar, line, pie, doughnut) to show prediction distribution, confidence trends, and model comparison.

Procedure

Step 1: Create Project Directory

```
mkdir exp6_dashboard cd
exp6_dashboard
```

Step 2: Create and Activate Virtual Environment

```
python -m venv venv
```

Windows:

```
venv\Scripts\activate
```

Mac/Linux:

```
source venv/bin/activate
```

Step 3: Install Required Packages

```
pip install flask pymongo python-dotenv scikit-learn
```

Step 4: Copy Model Files

Copy model files from previous experiments:

```
cp ../exp3_ml_training/*.pkl .
```

Step 5: Set Up MongoDB Atlas

Use the same MongoDB Atlas cluster from Experiment 2, or follow the setup steps from that experiment.

Step 6: Create .env File

```
MONGO_URI=mongodb+srv://admin:YOUR_PASSWORD@cluster0.xxxxx.mongodb.net/
```

> **Important**
>
> Above line is an example. Replace with the line from `.env` file in experiment 2.

Step 7: Create Project Structure

```
mkdir templates mkdir
static mkdir static/css
mkdir static/js
```

Step 8: Create app.py

```
from flask import Flask, render_template, request, jsonify from
pymongo import MongoClient
import pickle
import numpy as np
import os
from dotenv import load_dotenv
from datetime import datetime
from bson import json_util
import json

load_dotenv()

app = Flask(__name__)
app.secret_key = os.getenv('SECRET_KEY', 'default_secret_key')

# MongoDB connection
client =
MongoClient(os.getenv('MONGO_URI')) db =
client['fullstack_ai_lab'] predictions_collection
= db['predictions']

# Load models
with open('iris_logistic_regression.pkl', 'rb') as f:
```

```
    logistic_model = pickle.load(f)

naive_bayes_model = logistic_model

with open('feature_names.pkl', 'rb') as f:
    feature_names = pickle.load(f)
with open('target_names.pkl', 'rb') as f:
    target_names = pickle.load(f)

@app.route('/')
def home():
    return render_template('dashboard.html')

@app.route('/api/predict', methods=['POST']) def
predict():
    try:
        data = request.get_json() features
        = data.get('features')
        model_name = data.get('model', 'logistic_regression')

        if not features or len(features) != 4:
            return jsonify({'error': 'Expected 4 features'}), 400

        features_array = np.array([features])

        # Select model
        if model_name == 'logistic_regression':
            model = logistic_model
        elif model_name == 'naive_bayes':
            model = naive_bayes_model
        else:
            return jsonify({'error': 'Invalid model name'}), 400

        # Predict
        prediction = model.predict(features_array)
        prediction_index = int(prediction[0]) predicted_class
        = target_names[prediction_index]

        # Get probabilities
        probabilities = model.predict_proba(features_array)[0]
        prob_dict = {
```

```
            target_names[i]: float(probabilities[i]) for i
            in range(len(target_names))
        }

        # Store prediction in database
        prediction_record = {
            'timestamp': datetime.utcnow(),
            'model': model_name,
            'features': {
                feature_names[i]: features[i]
                for i in range(len(features))
            },
            'prediction': predicted_class,
            'prediction_index': prediction_index,
            'probabilities': prob_dict, 'confidence':
            float(max(probabilities))
        }

        predictions_collection.insert_one(prediction_record)

        response = {
            'model_used': model_name,
            'prediction': predicted_class,
            'prediction_index': prediction_index,
            'probabilities': prob_dict,
            'confidence': float(max(probabilities))
        }

        return jsonify(response)

    except Exception as e:
        return jsonify({'error': str(e)}), 500

@app.route('/api/stats', methods=['GET'])
def get_stats():
    """Get prediction statistics for dashboard"""
    try:
        # Total predictions
        total_predictions = predictions_collection.count_documents({})

        # Predictions by class
        pipeline = [
            {
```

```
            '$group': {
                '_id': '$prediction', 'count':
                {'$sum': 1}
            }
        }
]
predictions_by_class = list(predictions_collection.aggregate(pipeline))

# Predictions by model
pipeline = [
        {
            '$group': {
                '_id': '$model',
                'count': {'$sum': 1}
            }
        }
]
predictions_by_model = list(predictions_collection.aggregate(pipeline))

# Average confidence by model
pipeline = [
        {
            '$group': {
                '_id': '$model',
                'avg_confidence': {'$avg': '$confidence'}
            }
        }
]

avg_confidence_by_model = list(predictions_collection.aggregate(pipeline))

# Recent predictions (last 10)
recent_predictions = list(
        predictions_collection.find()
        .sort('timestamp', -1)
        .limit(10)
)

# Confidence distribution
pipeline = [
        {
            '$group': {
                '_id': {
                    '$cond': [
                        {'$gte': ['$confidence', 0.9]}, 'High (>90%)',
```

```
                    {'$cond': [
                        {'$gte': ['$confidence', 0.7]}, 'Medium (70-90%)',
                        'Low (<70%)'
                    ]}
                ]
            },
            'count': {'$sum': 1}
        }
    }
]
    confidence_distribution = list(predictions_collection.aggregate(pipeline))

    return json.loads(json_util.dumps({ 'total_predictions':
        total_predictions, 'predictions_by_class':
        predictions_by_class, 'predictions_by_model':
        predictions_by_model, 'avg_confidence_by_model':
        avg_confidence_by_model, 'recent_predictions':
        recent_predictions, 'confidence_distribution':
        confidence_distribution
    }))

except Exception as e:
    return jsonify({'error': str(e)}), 500

@app.route('/api/clear-history', methods=['POST']) def
clear_history():
    """Clear all prediction history"""
    try:
        result = predictions_collection.delete_many({})
        return jsonify({
            'success': True,
            'deleted_count': result.deleted_count
        })
    except Exception as e:
        return jsonify({'error': str(e)}), 500 if __

name_____ == '__main__':
```

```
app.run(debug=True, host='0.0.0.0', port=5000)
```

Step 9: Create templates/dashboard.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>ML Dashboard - Iris Predictor</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='css/dashboard.css') }}">
    <script src="https://cdn.jsdelivr.net/npm/chart.js@4.4.0/dist/chart.umd.min.js">
</script>
</head>
<body>
    <div class="dashboard">
        <header>
            <h1>□ ML Prediction Dashboard</h1>
            <p>Real-time visualization of iris flower predictions</p>
        </header>

        <div class="stats-grid">
            <div class="stat-card">
                <div class="stat-icon">□</div>
                <div class="stat-content">
                    <div class="stat-value" id="totalPredictions">0</div>
                    <div class="stat-label">Total Predictions</div>
                </div>
            </div>

            <div class="stat-card">
                <div class="stat-icon">□</div>
                <div class="stat-content">
                    <div class="stat-value" id="avgConfidence">0%</div>
                    <div class="stat-label">Avg Confidence</div>
                </div>
            </div>

            <div class="stat-card">
                <div class="stat-icon">□</div>
                <div class="stat-content">
                    <div class="stat-value" id="modelsUsed">0</div>
                    <div class="stat-label">Models Used</div>
                </div>
```

```
            </div>

            <div class="stat-card">
                <div class="stat-icon">?</div>
                <div class="stat-content">
                    <div class="stat-value" id="topPrediction">-</div>
                    <div class="stat-label">Most Common</div>
                </div>
            </div>

        </div>

        <div class="prediction-form-container">
            <h2>Make a Prediction</h2>
            <form id="predictionForm">
                <div class="input-row">
                    <input type="number" id="sepal_length" placeholder="Sepal Length (cm)"
                            step="0.1" required>
                    <input type="number" id="sepal_width" placeholder="Sepal Width (cm)"
                            step="0.1" required>
                    <input type="number" id="petal_length" placeholder="Petal Length (cm)"
                            step="0.1" required>
                    <input type="number" id="petal_width" placeholder="Petal Width (cm)"
                            step="0.1" required>
                </div>
                <div class="input-row">
                    <select id="model" required>
                        <option value="logistic_regression">Logistic Regression</option>
                        <option value="naive_bayes">Naive Bayes</option>
                    </select>
                    <button type="submit">Predict</button>
                </div>
            </form>
            <div id="predictionResult" style="display:none;"></div>
        </div>

        <div class="charts-grid">
            <div class="chart-container">
                <h3>Predictions by Species</h3>
                <canvas id="predictionsByClass"></canvas>
            </div>

            <div class="chart-container">
```

```
                <h3>Predictions by Model</h3>
                <canvas id="predictionsByModel"></canvas>
            </div>

            <div class="chart-container">
                <h3>Confidence Distribution</h3>
                <canvas id="confidenceDistribution"></canvas>
            </div>

            <div class="chart-container">
                <h3>Model Performance Comparison</h3>
                <canvas id="modelComparison"></canvas>
            </div>
        </div>

        <div class="recent-predictions">
            <div class="section-header">
                <h2>Recent Predictions</h2>
                <button id="clearHistory" class="btn-clear">Clear History</button>
            </div>
```

```
            <div id="recentPredictionsList"></div>
        </div>
    </div>

    <script src="{{ url_for('static', filename='js/dashboard.js') }}"></script>
</body>
</html>
```

Step 10: Create static/css/dashboard.css

```
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    background: #f5f7fa;
}

.dashboard {
    max-width: 1400px;
    margin: 0 auto;
    padding: 20px;
```

```
}

header {
    text-align: center;
    padding: 40px
    20px;
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    color: white;
    border-radius: 10px;
    margin-bottom:
    30px;
    box-shadow: 0 5px 15px rgba(0,0,0,0.1);
}

header h1 {
    font-size: 2.5em;
    margin-bottom:
    10px;
}

header p {
    font-size: 1.1em;
    opacity: 0.9;
}

.stats-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(250px, 1fr)); gap:
    20px;
    margin-bottom:  30px;
}

.stat-card {
    background: white;
    padding: 25px;
    border-radius: 10px;
    box-shadow: 0 2px 10px rgba(0,0,0,0.1);
    display: flex;
    align-items: center; gap:
    20px;
    transition: transform 0.3s;
}

.stat-card:hover  {
```

```
    transform:  translateY(-5px);
    box-shadow: 0 5px 20px rgba(0,0,0,0.15);
}

.stat-icon {
    font-size: 3em;
}

.stat-value {
    font-size: 2em;
    font-weight: bold;
    color: #667eea;
}

.stat-label {
    color: #666; font-
    size: 0.9em;
}

.prediction-form-container {
    background: white;
    padding: 30px;
    border-radius:  10px;
    box-shadow: 0 2px 10px rgba(0,0,0,0.1);
    margin-bottom: 30px;
}

.prediction-form-container h2 {
    color: #333;
    margin-bottom:  20px;
}

.input-row {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(200px, 1fr)); gap:
    15px;
    margin-bottom:  15px;
}

input, select {
```

```
    padding: 12px;
```

```
    border: 2px solid #ddd;
    border-radius: 5px;
    font-size: 14px;
}

input:focus, select:focus { outline:
    none;
    border-color:  #667eea;
}

button[type="submit"] {
    padding: 12px 30px;
    background:
    #667eea; color:
    white; border: none;
    border-radius: 5px;
    cursor: pointer; font-
    size: 16px;
    transition: background 0.3s;
}

button[type="submit"]:hover {
    background: #764ba2;
}

#predictionResult {
    margin-top: 20px;
    padding: 20px;
    background: #f0f8ff;
    border-radius: 5px;
    border-left: 4px solid #667eea;
}

.charts-grid { display:
    grid;
    grid-template-columns: repeat(auto-fit, minmax(400px, 1fr)); gap:
    20px;
    margin-bottom:  30px;
}

.chart-container {
    background: white;
    padding: 25px;
```

```
    border-radius: 10px;
    box-shadow: 0 2px 10px rgba(0,0,0,0.1);
}

.chart-container h3 {
    color: #333;
    margin-bottom:
    20px; text-align:
    center;
}


.chart-container canvas {
    max-height: 300px;
}

.recent-predictions {
    background: white;
    padding: 30px;
    border-radius: 10px;
    box-shadow: 0 2px 10px rgba(0,0,0,0.1);
}

.section-header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom:  20px;
}

.section-header h2 {
    color: #333;
}

.btn-clear {
    padding: 10px 20px;
    background:
    #dc3545; color:
    white; border: none;
    border-radius: 5px;
    cursor: pointer;
    transition: background 0.3s;
}
```

```
.btn-clear:hover {
    background: #c82333;
}

.prediction-item {
    padding: 15px;
    border-bottom: 1px solid #eee;
    display: grid;
    grid-template-columns: 1fr 2fr 1fr 1fr;
    gap: 15px;
    align-items:  center;
}

.prediction-item:last-child {
    border-bottom: none;
}

.prediction-time {
    color: #999; font-
    size: 0.9em;
```

```
}

.prediction-class { font-weight:
        bold; color: #667eea;
        text-transform:  capitalize;
}

.prediction-model { color:
        #666; font-size:0.9em;
}

.prediction-confidence { text-align:
        right; font-weight: bold;
        color: #28a745;
}

@media (max-width: 768px) {
        .charts-grid  {
                grid-template-columns:  1fr;
        }

        .prediction-item  {
                grid-template-columns: 1fr; gap: 5px;
        }
}
```

Step 11: Create static/js/dashboard.js

```
let charts = {};

// Load dashboard data on page load document.addEventListener('DOMContentLoaded', function()
{
        loadDashboardData();

        // Set up prediction form document.getElementById('predictionForm').addEventListener('submit',  handlePrediction);

        // Set up clear history button document.getElementById('clearHistory').addEventListener('click', clearHistory);

        // Refresh data every 30 seconds
        setInterval(loadDashboardData, 30000);
});

async function loadDashboardData() { try {
                const response=await fetch('/api/stats'); const data = await
                response.json();
```

```
        updateStatCards(data);
        updateCharts(data);
        updateRecentPredictions(data.recent_predictions);
    } catch (error) {
        console.error('Error loading dashboard data:', error);
    }
}

function updateStatCards(data) {
    // Total predictions
    document.getElementById('totalPredictions').textContent =
        data.total_predictions || 0;

    // Average confidence
    if (data.avg_confidence_by_model && data.avg_confidence_by_model.length >
        0) { const avgConf = data.avg_confidence_by_model.reduce((sum, item) =>
            sum + item.avg_confidence, 0) / data.avg_confidence_by_model.length;
        document.getElementById('avgConfidence').textContent =
            (avgConf * 100).toFixed(1) + '%';
    }

    // Models used
    document.getElementById('modelsUsed').textContent =
        data.predictions_by_model ? data.predictions_by_model.length : 0;

    // Most common prediction
    if (data.predictions_by_class && data.predictions_by_class.length > 0) { const
        topPred = data.predictions_by_class.reduce((max, item) =>
            item.count > max.count ? item : max, data.predictions_by_class[0]);
        document.getElementById('topPrediction').textContent =
            topPred._id.charAt(0).toUpperCase() + topPred._id.slice(1);
    }
}

function updateCharts(data) {
    // Predictions by Class
    if (data.predictions_by_class) {
        const labels = data.predictions_by_class.map(item => item._id.charAt(0).toUpperCase()
            + item._id.slice(1));
        const counts = data.predictions_by_class.map(item => item.count);

        createOrUpdateChart('predictionsByClass', {
```

```
            type: 'bar',
            data: {
                labels: labels, datasets:
                [{
                    label: 'Count',
                    data: counts,
                    backgroundColor:
                    [
                        'rgba(102, 126, 234, 0.8)',
                        'rgba(118, 75, 162, 0.8)',
                        'rgba(237, 100, 166, 0.8)'
                    ]









                }]
            },
            options: {
                responsive: true,
                maintainAspectRatio: true,
                plugins: {
                    legend: { display: false }
                }
            }
        });
    }

    // Predictions by Model
    if (data.predictions_by_model) {
        const labels = data.predictions_by_model.map(item =>
            item._id.replace('_', ' ').replace(/\b\w/g, l => l.toUpperCase()));
        const counts = data.predictions_by_model.map(item => item.count);

        createOrUpdateChart('predictionsByModel', {
            type: 'doughnut',
            data: {
                labels: labels, datasets:
                [{
                    data: counts,
                    backgroundColor:
                    [
```

```
                    'rgba(102, 126, 234, 0.8)',
                    'rgba(118, 75, 162, 0.8)'
                ]
            }]
        },
        options: {
            responsive: true,
            maintainAspectRatio: true
        }
    });
}

// Confidence Distribution
if (data.confidence_distribution) {
    const labels = data.confidence_distribution.map(item => item._id); const
    counts = data.confidence_distribution.map(item => item.count);

    createOrUpdateChart('confidenceDistribution', {
        type: 'pie',
        data: {
            labels: labels, datasets:
            [{
                data: counts,
                backgroundColor:
                [
                    'rgba(40, 167, 69, 0.8)',
                    'rgba(255, 193, 7, 0.8)',
                    'rgba(220, 53, 69, 0.8)'
                ]
```

```
                }]
            },
            options: {
                responsive: true,
                maintainAspectRatio: true
            }
        });
    }

    // Model Performance Comparison
    if (data.avg_confidence_by_model) {
        const labels = data.avg_confidence_by_model.map(item =>
            item._id.replace('_', ' ').replace(/\b\w/g, l => l.toUpperCase()));
        const avgConf = data.avg_confidence_by_model.map(item =>
            (item.avg_confidence * 100).toFixed(2));

        createOrUpdateChart('modelComparison', {
            type: 'bar',
            data: {
                labels: labels, datasets:
                [{
                    label: 'Average Confidence (%)',
                    data: avgConf,
                    backgroundColor: 'rgba(102, 126, 234, 0.8)'
                }]
            },
            options: {
                responsive: true,
                maintainAspectRatio: true,
                scales: {
                    y: {
                        beginAtZero: true,
                        max: 100
                    }
                }
            }
        });
    }
}

function createOrUpdateChart(canvasId, config) {
    const ctx = document.getElementById(canvasId);

    if (charts[canvasId]) {
```

```
        charts[canvasId].destroy();
    }

    charts[canvasId] = new Chart(ctx, config);
}

function updateRecentPredictions(predictions) {
    const container = document.getElementById('recentPredictionsList');

    if (!predictions || predictions.length === 0) {
        container.innerHTML = '<p style="text-align:center;color:#999;">No predictions
yet</p>';
        return;
    }

    container.innerHTML = '';

    predictions.forEach(pred => {
        const item = document.createElement('div');
        item.className = 'prediction-item';

        const timestamp = new Date(pred.timestamp.$date || pred.timestamp);
        const timeStr = timestamp.toLocaleString();

        item.innerHTML = `
            <div class="prediction-time">${timeStr}</div>
            <div class="prediction-class">${pred.prediction}</div>
            <div class="prediction-model">${pred.model.replace('_', ' ')}</div>
            <div class="prediction-confidence">${(pred.confidence * 100).toFixed(1)}%</div>
        `;

        container.appendChild(item);
    });
}

async function handlePrediction(e) {
    e.preventDefault();

    const features = [
        parseFloat(document.getElementById('sepal_length').value),
        parseFloat(document.getElementById('sepal_width').value),
        parseFloat(document.getElementById('petal_length').value),
        parseFloat(document.getElementById('petal_width').value)
    ];
```

```
const model = document.getElementById('model').value;

try {
    const response = await fetch('/api/predict', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ features, model })
    });

    const data = await response.json(); if

    (response.ok) {
        const resultDiv = document.getElementById('predictionResult');
        resultDiv.style.display = 'block';
        resultDiv.innerHTML = `
            <strong>Prediction:</strong> ${data.prediction.toUpperCase()}<br>
            <strong>Confidence:</strong> ${(data.confidence * 100).toFixed(2)}%<br>
```

```
            <strong>Model:</strong> ${data.model_used.replace('_', ' ')}
        `;

        // Refresh dashboard after prediction setTimeout(loadDashboardData, 500);
    }
} catch (error) {
    alert('Prediction failed: ' + error.message);
}
}

async function clearHistory() {
    if (!confirm('Are you sure you want to clear all prediction history?')) { return;
    }

    try {
        const response = await fetch('/api/clear-history', { method: 'POST'
        });

        const data = await response.json(); if (data.success) {
            alert(`Cleared ${data.deleted_count} predictions`); loadDashboardData();
        }
    } catch (error) {
        alert('Failed to clear history: ' + error.message);
    }
}
```

Step 12: Run the Application

```
python app.py
```

Step 13: Test the Dashboard

1. Open browser at `http://localhost:5000`
2. Make several predictions using different inputs and models
3. Watch the charts update in real-time
4. Observe the statistics cards updating
5. Check the recent predictions list
6. Try clearing the history

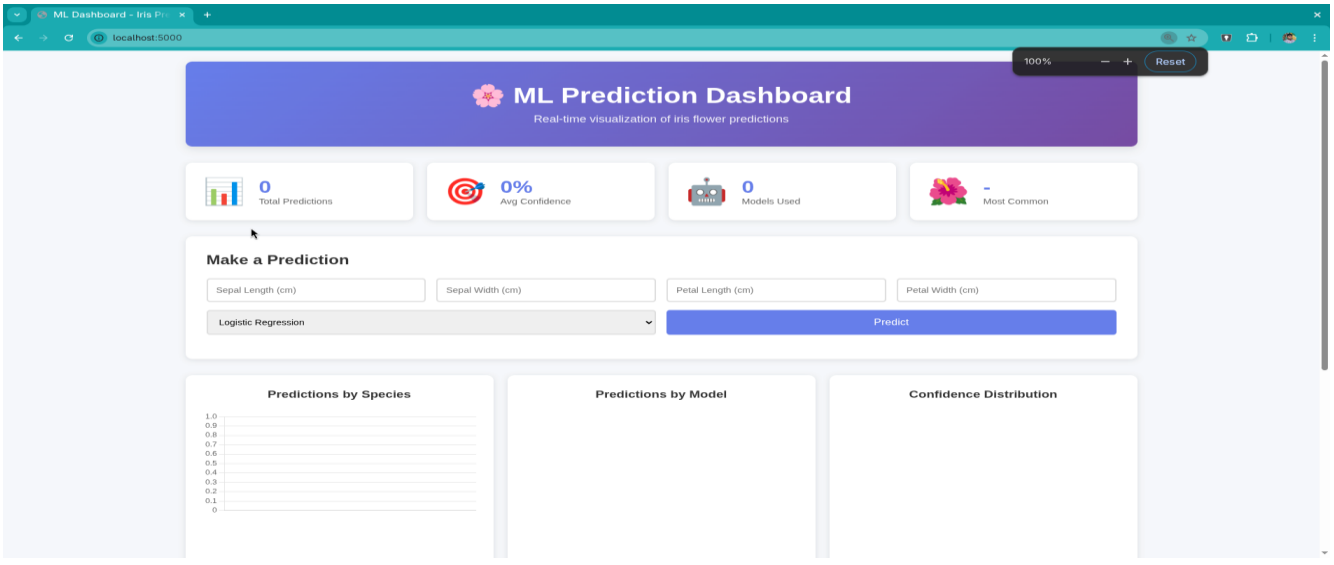
Output

A comprehensive dashboard with:

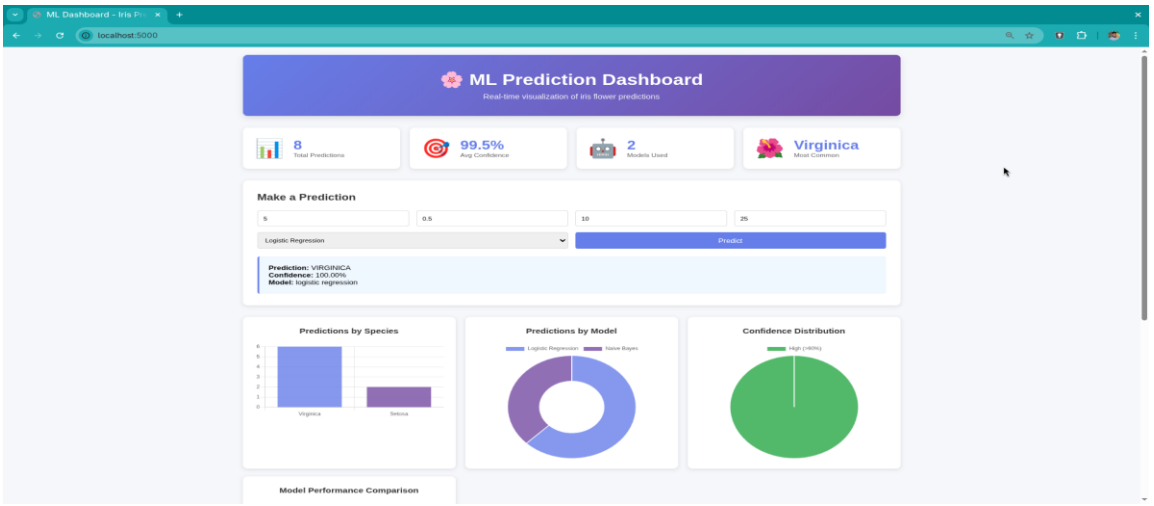
- Real-time statistics (total predictions, average confidence, etc.)
- Four interactive Chart.js visualizations:
  - Bar chart showing predictions by species
  - Doughnut chart showing model usage
  - Pie chart showing confidence distribution
  - Bar chart comparing model performance
- Live prediction form integrated into dashboard
- Recent predictions list with timestamps
- Auto-refresh every 30 seconds
- All prediction data stored in MongoDB

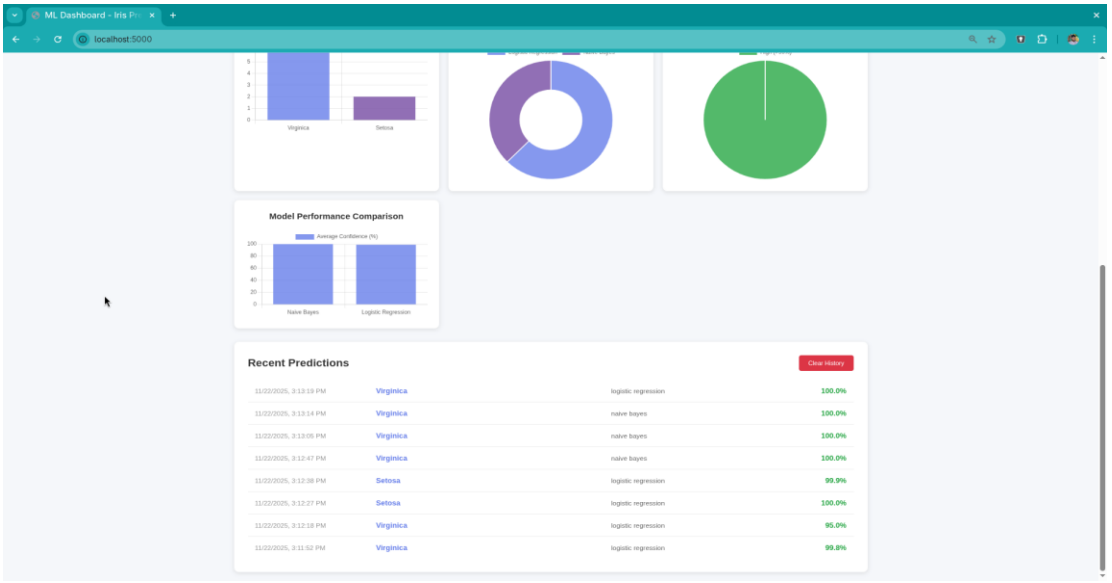
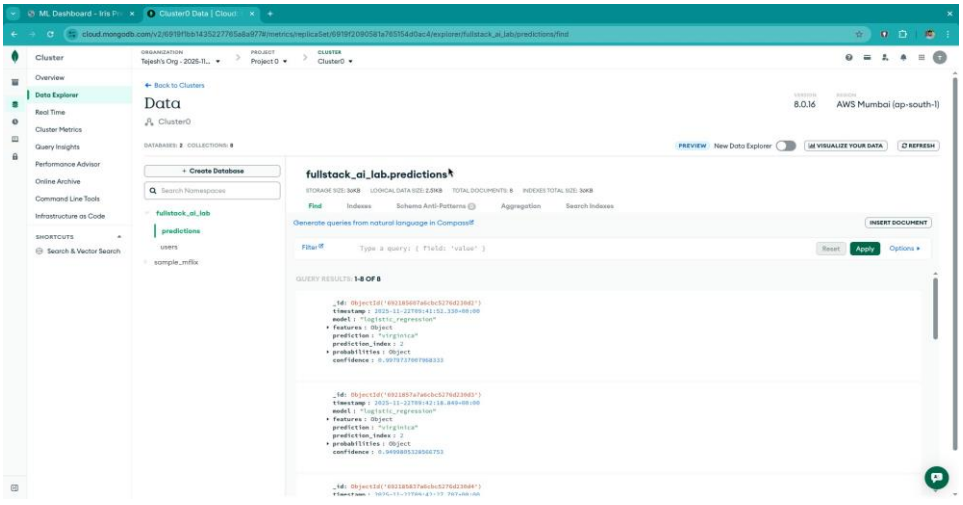
Screen-shots

Default page:



After Predictions





Values shown in MongoDB Atlas